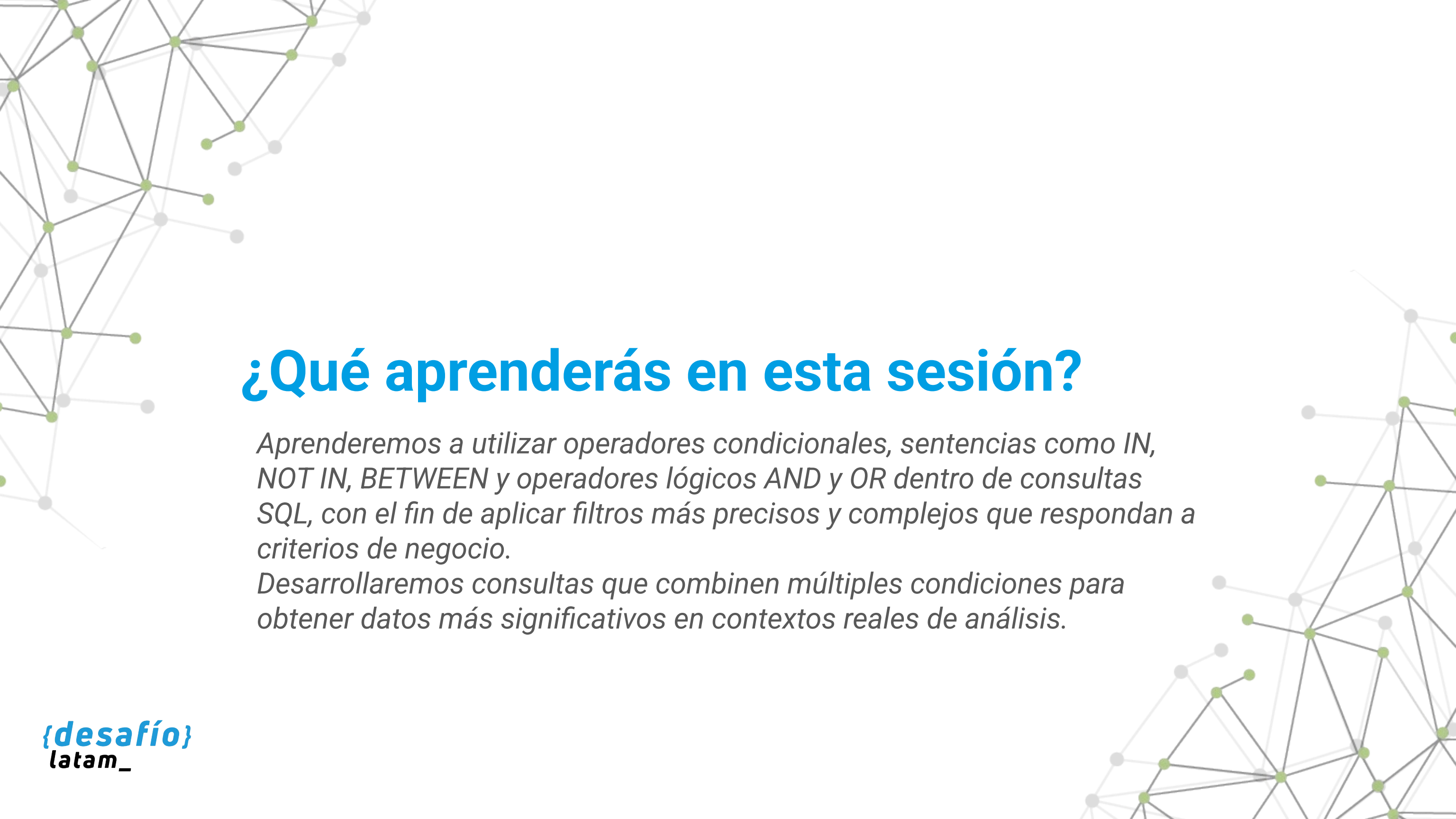




Fundamentos de Bases de Datos y Consultas Básicas

Clase 5 - Operadores y condiciones en SQL



¿Qué aprenderás en esta sesión?

Aprenderemos a utilizar operadores condicionales, sentencias como IN, NOT IN, BETWEEN y operadores lógicos AND y OR dentro de consultas SQL, con el fin de aplicar filtros más precisos y complejos que respondan a criterios de negocio.

Desarrollaremos consultas que combinen múltiples condiciones para obtener datos más significativos en contextos reales de análisis.

Desafío inicial

Eres parte del equipo de análisis de datos de una empresa de telecomunicaciones que ofrece servicios a nivel nacional. Los distintos departamentos te han solicitado reportes segmentados para apoyar decisiones estratégicas:

- El área de soporte necesita identificar reclamos generados en regiones específicas durante los últimos 60 días.
- El área comercial busca conocer a las personas clientas con facturación superior a \$100.000 en las ciudades más grandes del país.
- El área de operaciones quiere excluir de sus análisis a ciertos tipos de servicios que están en proceso de eliminación.

Hasta ahora, los filtros aplicados han sido demasiado generales, lo que ha llevado a conclusiones inexactas. Para resolver este problema, deberás aplicar operadores condicionales y lógicos que te permitan combinar múltiples criterios, construir rangos, incluir o excluir registros específicos y segmentar con precisión los datos relevantes.

A lo largo de esta sesión aprenderás a estructurar consultas que respondan a estos requerimientos con exactitud, eficiencia y claridad técnica, preparando reportes útiles y accionables para las diferentes áreas de tu organización.

Operadores condicionales

¿Cuáles son las herramientas que nos permiten comparar valores en una consulta SQL y tomar decisiones basadas en esos resultados?

Los operadores condicionales permiten establecer comparaciones entre valores dentro de una consulta SQL. Son fundamentales para aplicar criterios de filtrado dentro de cláusulas como WHERE, y permiten construir expresiones que devuelven verdadero o falso, determinando si un registro debe incluirse o no en los resultados. Estos operadores son ampliamente utilizados en contextos productivos donde la toma de decisiones depende de la exactitud de los filtros aplicados a los datos.

Comparaciones básicas

Los operadores condicionales más comunes son:

Operador	Significado	Ejemplo
=	Igual a	precio = 1000
!= o <>	Distinto de	estado <> 'Cancelado'
>	Mayor que	edad > 18
<	Menor que	fecha < '2024-01-01'
>=	Mayor o igual que	sueldo >= 500000
<=	Menor o igual que	puntos <= 80

Tabla 1. Operadores condicionales básicos en SQL

Fuente: Desafío Latam

Consideraciones avanzadas sobre comparaciones en SQL

En esta sección abordamos escenarios más complejos y avanzados en el uso de condiciones dentro de consultas SQL, especialmente útiles para analistas que deben enfrentar desafíos en entornos de datos heterogéneos y con múltiples reglas de negocio.

Comparaciones alfanuméricas

SQL permite comparar valores alfabéticos como si fueran numéricos en términos de orden. Por ejemplo, se puede ejecutar:

```
SELECT *  
FROM clientes  
WHERE apellido > 'M';
```

Esta consulta retorna los registros donde el apellido comienza con letras posteriores a la M. Este tipo de operación puede resultar útil para paginar resultados o establecer particiones por rango alfabético. Sin embargo, es importante tener en cuenta que el orden puede estar influenciado por el collation de la base de datos (configuración regional que afecta el ordenamiento y sensibilidad a mayúsculas/minúsculas).

Comparaciones con fechas

Uno de los grandes beneficios de SQL es que permite trabajar con fechas de forma directa. Por ejemplo:

```
SELECT * FROM clientes WHERE fecha_registro >= '2024-07-01';
```

Este tipo de comparación es muy útil para filtrar datos por rangos temporales, realizar cortes mensuales o calcular métricas históricas. Es importante validar que las fechas estén correctamente almacenadas como tipo DATE o TIMESTAMP, y que el formato sea compatible con el motor SQL utilizado.

Comparaciones con expresiones

SQL también permite realizar comparaciones complejas utilizando expresiones aritméticas o funciones. Por ejemplo:

```
SELECT *  
FROM ventas  
WHERE total > (cantidad * precio_unitario);
```

Este tipo de consulta es útil en contextos comerciales o financieros, donde se requiere validar que los registros cumplan con una regla de negocio calculada en tiempo real. Estas condiciones también pueden integrarse a cláusulas HAVING para evaluar resultados agregados.

Aplicación práctica en contextos productivos

En escenarios reales, estos operadores se utilizan para filtrar información que cumpla ciertos criterios de negocio. Por ejemplo, si una empresa desea obtener una lista de clientas con deudas superiores a \$100.000:

```
SELECT
    nombre,
    deuda
FROM clientes
WHERE deuda > 100000;
```

En un entorno financiero, esta consulta permite priorizar los esfuerzos de cobranza en los casos con mayor impacto monetario. Del mismo modo, una consulta como:

```
SELECT
    nombre,
    ingreso_mensual
FROM postulantes
WHERE ingreso_mensual >= 1200000;
```

Podría ser utilizada por una entidad bancaria para prefiltrar clientas aptas para una línea de crédito.

Buenas prácticas

1

Validar los tipos de datos

Asegúrate de que las comparaciones se realicen entre tipos compatibles. Comparar VARCHAR con INTEGER puede generar resultados erróneos o vacíos.

2

Usar alias de columna cuando sea necesario

Esto mejora la legibilidad, especialmente cuando se trabaja con varias tablas o subconsultas.

3

Mantener consistencia en las comparaciones

En especial cuando se utilizan varios filtros con condiciones similares, evitar redundancias mejora el rendimiento y claridad del código.

4

Revisar índices en columnas filtradas

Optimiza el tiempo de respuesta en bases de datos grandes.

5

Comentar filtros complejos

Cuando se usen múltiples condiciones, los comentarios ayudan a mantener la mantenibilidad del código.

Errores comunes

Tratar NULL como un valor común

En SQL, NULL representa un valor desconocido. Comparaciones como WHERE campo = NULL siempre devolverán falso. Debe utilizarse IS NULL o IS NOT NULL.

Omisión de comillas en textos

Usar WHERE ciudad = Santiago puede causar errores. Debe ser WHERE ciudad = 'Santiago'.

Uso incorrecto de comparaciones con fechas

Si el formato no es válido o no se ajusta al motor de base de datos, las comparaciones pueden fallar silenciosamente.

No considerar zonas horarias

En campos TIMESTAMP, comparar sin ajustar zonas puede excluir resultados válidos.

Comparar campos que pueden contener NULL sin control

Esto puede provocar que resultados importantes no se muestren. Es buena práctica combinar condiciones con IS NOT NULL cuando se desea evitarlo.

Dominar las comparaciones individuales es solo el primer paso. En muchas situaciones reales necesitamos trabajar con listas de valores o rangos definidos, como ciudades prioritarias, períodos de fechas o clasificaciones específicas. Para ello, exploraremos herramientas más compactas y eficientes como las sentencias IN, NOT IN y BETWEEN, que permiten una mayor expresividad al construir filtros compuestos en nuestras consultas SQL.

Sentencias IN, NOT IN, BETWEEN

¿Cómo podrías filtrar eficientemente registros que coincidan con múltiples valores o que se encuentren dentro de un rango definido sin escribir múltiples condiciones una por una?

Estas sentencias permiten aplicar filtros más compactos y eficientes. Son especialmente útiles cuando se requiere filtrar por listas de valores o por intervalos específicos, lo que evita repetir múltiples condiciones con operadores lógicos como OR o AND.

Uso de IN y NOT IN

La cláusula IN permite verificar si un valor se encuentra dentro de una lista específica de valores. NOT IN evalúa lo contrario: que el valor no esté dentro del conjunto listado.

```
SELECT
    nombre,
    ciudad
FROM clientes
WHERE ciudad IN ('Santiago', 'Valparaíso', 'Concepción');
```

Este ejemplo selecciona a todas las personas clientas que viven en las ciudades mencionadas, sin necesidad de escribir múltiples condiciones con OR.

Para excluir ciudades:

```
WHERE ciudad != 'Arica'
AND ciudad != 'Punta Arenas'
```

Aplicación productiva



Identificar sucursales prioritarias

Identificar sucursales ubicadas solo en zonas prioritarias.



Excluir productos

Excluir productos que ya no están disponibles en campañas activas.



Filtrar personal

Filtrar registros de personal activo en áreas específicas del negocio.



Clasificar campañas

Clasificar campañas de marketing dirigidas a segmentos predefinidos.

Buenas prácticas

- Asegúrate de que los elementos en la lista coincidan en tipo y formato con los del campo a evaluar.
- Evita listas muy largas en consultas sobre grandes volúmenes, ya que puede afectar el rendimiento. Considera el uso de subconsultas con SELECT anidados.
- Cuando sea posible, mantén las listas dinámicas extrayéndolas desde otras tablas. Ejemplo:

```
SELECT
    nombre,
    ciudad
FROM clientes
WHERE ciudad IN (
    SELECT ciudad
    FROM zonas_prioritarias
);
```



Errores comunes

- ⚠ • Incluir NULL dentro de una lista en IN puede provocar resultados inesperados si no se gestiona con IS NULL explícitamente.
- Usar cadenas de texto sin comillas simples: 'Valparaíso' y no Valparaíso.
- Comparar con listas numéricas y usar comillas innecesarias (por ejemplo, IN ('1';2) para campo INTEGER).

Uso de BETWEEN

La sentencia BETWEEN permite seleccionar valores dentro de un rango, incluyendo los extremos.

```
SELECT
    nombre,
    edad
FROM empleados
WHERE edad BETWEEN 30 AND 40;
```

Este filtro recupera a las personas cuya edad esté entre 30 y 40 años, incluidos ambos valores.

Aplicación productiva



Seleccionar facturación trimestral

Seleccionar facturación en un trimestre específico (fecha BETWEEN '2024-01-01' AND '2024-03-31').



Filtrar inventario por precios

Filtrar inventario según rangos de precios.



Identificar clientes por consumo

Identificar clientes con consumo mensual dentro de un umbral de interés comercial.



Evaluar desempeños

Evaluar desempeños que estén dentro de rangos de rendimiento aceptables en un dashboard de desempeño.

Comparación de uso de operadores múltiples vs. BETWEEN

Sintaxis tradicional	Sintaxis con BETWEEN
WHERE edad >= 30 AND edad <= 40	WHERE edad BETWEEN 30 AND 40

Tabla 2. Ejemplo de equivalencia entre sintaxis estándar y BETWEEN

Fuente: Desafío Latam

Buenas prácticas



Verificar que el campo comparado tenga un tipo de dato numérico, fecha o texto alfabético ordenable.



En fechas, confirmar el formato correcto y los valores extremos deseados.



Usar **BETWEEN** para mejorar la legibilidad de filtros por rangos, especialmente en reportes operativos recurrentes.



En estructuras de consulta automatizadas, documentar con **comentarios los rangos utilizados**, para facilitar ajustes futuros.

Errores comunes

Invertir los valores del rango

BETWEEN 40 AND 30 genera una consulta válida, pero sin resultados.

No considerar valores NULL

No considerar si el campo contiene NULL, lo cual excluye automáticamente esos registros a menos que se complemente con OR campo IS NULL.

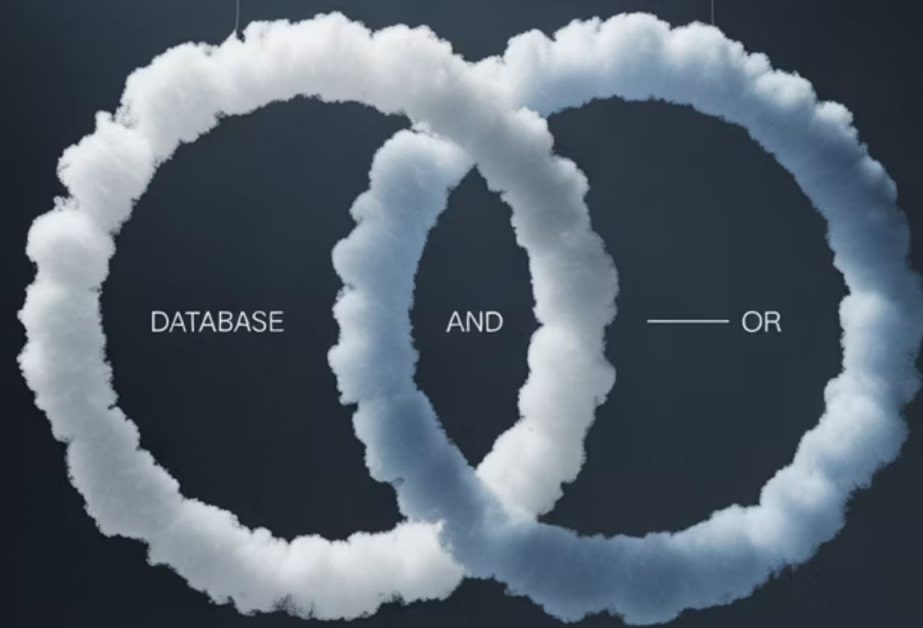
No revisar el tipo de dato

Aplicar BETWEEN sobre campos VARCHAR puede generar ordenamientos inesperados, especialmente con valores alfabéticos mixtos o inconsistentes.

Problemas con timestamps

Utilizar BETWEEN en campos con timestamps sin truncar horas/minutos puede excluir datos deseados si no se ajusta correctamente el rango.

Habiendo revisado formas eficientes de filtrar listas y rangos mediante IN, NOT IN y BETWEEN, podemos ahora avanzar hacia el uso combinado de condiciones a través de los operadores lógicos AND y OR. Estos nos permitirán construir filtros complejos con múltiples criterios que simulan razonamientos más cercanos a los del análisis humano.



Operadores lógicos AND y OR

¿Qué sucede cuando necesitas aplicar más de una condición para filtrar los datos en SQL?, ¿Cómo controlas que se cumplan simultáneamente o de forma alternativa?

Cuando trabajamos con filtros compuestos, los operadores lógicos AND y OR permiten combinar múltiples expresiones condicionales en una misma consulta. Estas combinaciones son esenciales para crear filtros más precisos y obtener resultados más representativos de la realidad operativa.

Uso de AND

El operador AND exige que se cumplan todas las condiciones especificadas.

```
SELECT
    nombre,
    ciudad,
    edad
FROM clientes
WHERE ciudad = 'Santiago'
    AND edad >= 30;
```

Este ejemplo selecciona a las personas clientas que viven en Santiago y que tienen 30 años o más. Si una de las condiciones no se cumple, el registro se excluye del resultado.

Aplicación productiva

Identificar personal específico

Identificar personal de un área específica que supere un umbral de antigüedad.

Filtrar registros de ventas

Filtrar registros de ventas hechas por un vendedor determinado en un rango de fechas específico.

Aplicar segmentaciones de mercado

Aplicar segmentaciones de mercado que requieren múltiples atributos (ej. edad, región, ingresos).

Buenas prácticas y errores comunes

Buenas prácticas

- Asegúrate de utilizar paréntesis si mezclas AND con otros operadores lógicos para evitar ambigüedades.
- Ordena las condiciones de forma que las más restrictivas se evalúen primero en bases grandes (puede mejorar rendimiento en algunos motores SQL).

Errores comunes

- No utilizar paréntesis correctamente cuando se combinan múltiples condiciones con OR, lo que puede provocar filtros incorrectos.
- Usar = en lugar de >=, <=, etc., limitando innecesariamente la consulta.

Uso de OR

El operador OR permite que al menos una de las condiciones especificadas se cumpla.

```
SELECT
    nombre,
    ciudad,
    edad
FROM clientes
WHERE ciudad = 'Valparaíso'
    OR ciudad = 'Concepción';
```

Este filtro devuelve todos los registros cuya ciudad sea Valparaíso o Concepción.

Aplicación productiva



- Incluir en un reporte todas las regiones donde hay cobertura parcial o experimental.
- Construir consultas que agrupan varios segmentos similares (ej. productos en promoción, categorías prioritarias).

Buenas prácticas y errores comunes

Buenas prácticas

- Cuando uses múltiples OR, considera reemplazarlos por IN si se refiere a valores de una misma columna.
- Documenta qué representa cada grupo de condiciones en reportes críticos, para facilitar revisión futura.

Errores comunes

- Usar OR entre condiciones que podrían ser mutuamente excluyentes y generar ambigüedad o duplicidad.
- Esperar que OR y AND se evalúen de izquierda a derecha sin considerar precedencia de operadores.

Combinación de AND y OR

Muchas veces es necesario construir filtros más complejos, donde algunas condiciones deben cumplirse de forma obligatoria (AND) y otras de forma opcional (OR). En estos casos, es fundamental utilizar correctamente los paréntesis para agrupar las condiciones según el orden lógico deseado.

```
SELECT
    nombre,
    ciudad,
    edad
FROM clientes
WHERE (ciudad = 'Santiago' OR ciudad = 'Valparaíso')
    AND edad >= 30;
```

Este ejemplo selecciona a personas clientas que viven en Santiago o Valparaíso, y que además tienen 30 años o más. Sin los paréntesis, SQL interpretaría la consulta de forma diferente.

Aplicación productiva y diferencias en agrupación lógica

Aplicación productiva

- Identificar usuarios de regiones específicas que cumplan ciertos criterios de riesgo o oportunidad.
- Crear reglas de negocio para asignación automática de casos (por ejemplo, si región = norte o nivel consumo alto y servicio contratado = "premium").

Diferencias prácticas en agrupación lógica

Consulta sin paréntesis	Consulta con paréntesis correcta	Resultado esperado
A OR B AND C	(A OR B) AND C	Filtra correctamente

Tabla 3. Comparación de agrupación lógica con y sin paréntesis.

Fuente: Desafío Latam

Buenas prácticas y errores comunes

Buenas prácticas

- Siempre que combines AND y OR, usa paréntesis para indicar el orden lógico deseado.
- Revisa la lógica de negocio antes de escribir la consulta: ¿cuál condición es prioritaria?, ¿cuál es excluyente?

Errores comunes

- No agrupar correctamente las condiciones, lo que lleva a consultas que técnicamente se ejecutan, pero dan resultados incorrectos o incompletos.
- Repetir columnas innecesariamente dentro de combinaciones AND/OR, generando confusión o redundancia.

Actividad guiada:

Segmentando clientes para reportes estratégicos



Objetivo de la actividad

Aplicar operadores condicionales y lógicos (=, >, <, IN, NOT IN, BETWEEN, AND, OR) para filtrar registros específicos en una base de datos de clientes, generando consultas SQL que den respuesta a necesidades reales del área comercial, soporte y operaciones.



Situación problema

Trabajas en el área de inteligencia de negocios de una empresa de telecomunicaciones. Se te ha solicitado preparar tres reportes ejecutivos con información filtrada desde la base de datos de clientes. Las áreas interesadas han indicado los siguientes criterios:

Comercial

Clientes que viven en las ciudades de Santiago, Antofagasta o Concepción, y cuya facturación supere los \$100.000.

Soporte

Reclamos presentados durante los últimos 60 días, excluyendo clientes con servicios de tipo "prueba".

Operaciones

Listado de clientes entre 30 y 45 años, con servicios activos distintos a "migración" o "cancelación".

Para responder a estas solicitudes, deberás construir consultas SQL que integren múltiples condiciones y operadores de filtrado, priorizando la claridad, precisión y eficiencia.

Instrucciones

Crear la tabla base de clientes:

```
CREATE TABLE clientes (  
  id INT,  
  nombre VARCHAR(50),  
  ciudad VARCHAR(50),  
  edad INT,  
  servicio VARCHAR(50),  
  facturacion INT,  
  fecha_reclamo DATE  
);
```

Insertar datos de prueba:

```
INSERT INTO clientes VALUES  
(1, 'Ana', 'Santiago', 35, 'Internet', 120000, '2024-07-01'),  
(2, 'Luis', 'Valparaíso', 29, 'TV', 85000, '2024-06-15'),  
(3, 'Marcela', 'Concepción', 40, 'Internet', 130000, '2024-07-10'),  
(4, 'Diego', 'Antofagasta', 32, 'Prueba', 75000, '2024-05-30'),  
(5, 'Camila', 'Santiago', 45, 'Migración', 110000, '2024-06-20');
```



Ejecutar las siguientes consultas

a. Reporte comercial:

```
SELECT *  
FROM clientes  
WHERE ciudad IN ('Santiago', 'Antofagasta',  
  'Concepción')  
  AND facturacion > 100000;
```

b. Reporte de soporte:

```
SELECT *  
FROM clientes  
WHERE fecha_reclamo >= DATE('2024-07-18') -  
INTERVAL 60 DAY  
  AND servicio NOT IN ('Prueba');
```

c. Reporte de operaciones:

```
SELECT *  
FROM clientes  
WHERE edad BETWEEN 30 AND 45  
  AND servicio NOT IN ('Migración',  
  'Cancelación');
```

Comparar los resultados con las condiciones dadas. Discutir: ¿las consultas cumplen los objetivos?, ¿hay riesgos de excluir información útil?, ¿cómo mejorarían la redacción de los filtros?

Actividad práctica autónoma:

**Segmentación de datos de
clientes**



Objetivo de la actividad

Aplicar de forma autónoma operadores condicionales y lógicos para construir filtros en consultas SQL, respondiendo a criterios de segmentación en una base de datos simulada de clientes.

Contexto

Formas parte del equipo de análisis de datos de una empresa que administra una plataforma digital de servicios. Se te ha entregado una tabla de clientes con atributos clave como ciudad, edad, tipo de servicio y facturación.

Tu tarea es generar distintas consultas SQL que permitan segmentar esta base según condiciones específicas, justificando las decisiones tomadas.



Instrucciones para desarrollar la actividad de forma autónoma

Crea la tabla clientes con la siguiente estructura:

```
CREATE TABLE clientes (  
  id INT,  
  nombre VARCHAR(50),  
  ciudad VARCHAR(50),  
  edad INT,  
  servicio VARCHAR(50),  
  facturacion INT,  
  fecha_registro DATE  
);
```

Inserta los siguientes datos:

```
INSERT INTO clientes VALUES  
(1, 'Pedro', 'Santiago', 22, 'Streaming', 95000, '2024-05-01'),  
(2, 'Laura', 'Valparaíso', 30, 'Nube', 110000, '2024-06-10'),  
(3, 'Ignacio', 'Antofagasta', 27, 'Hosting', 105000, '2024-07-02'),  
(4, 'Fernanda', 'Concepción', 25, 'Streaming', 87000, '2024-07-15'),  
(5, 'Javiera', 'Santiago', 35, 'Prueba', 75000, '2024-06-20');
```



Genera las siguientes consultas

- Clientes con facturación superior a \$100.000.
- Clientes registrados en junio o julio de 2024.
- Clientes entre 25 y 35 años que no tengan servicio de tipo "Prueba".
- Clientes en ciudades distintas a "Valparaíso" o "Antofagasta".

Justifica en una tabla las decisiones de filtrado realizadas:

Consulta	Operadores usados	Justificación
Facturación > 100.000	>	Segmentación por alto valor de cliente
Fecha en junio/julio	BETWEEN, MONTH()	Período específico de análisis
Edad + tipo servicio	BETWEEN, NOT IN	Filtro combinado por rango y exclusión
Exclusión de ciudades	NOT IN	Focalización geográfica excluyente

Resumen

Operadores de comparación

Comprendimos cómo utilizar operadores de comparación para filtrar datos de forma precisa en consultas SQL.

1

2

Sentencias flexibles

Aplicamos sentencias IN, NOT IN, y BETWEEN para crear filtros más flexibles.

Operadores lógicos

Utilizamos operadores lógicos AND y OR para construir condiciones compuestas.

3

4

Buenas prácticas

Conocimos buenas prácticas y errores comunes al usar comparaciones alfanuméricas, con fechas y expresiones aritméticas.

Escenarios reales

Abordamos escenarios reales mediante actividades guiadas y autónomas.

5



Próxima sesión...

En la siguiente sesión abordaremos el tema: Funciones y operaciones de conjunto en SQL.

Aprenderemos a utilizar funciones preconstruidas como MIN, MAX, AVG, COUNT para generar estadísticas básicas directamente desde consultas SQL.

También exploraremos la sentencia DISTINCT para evitar duplicados y las operaciones UNION, INTERSECT y EXCEPT para combinar o contrastar resultados de múltiples consultas, ampliando así nuestra capacidad de análisis sobre grandes volúmenes de datos.

Preguntas de cierre

1

Diferencia entre IN y OR

¿Qué diferencia hay entre usar IN y una serie de condiciones con OR?

2

Validación de tipos de datos

¿Por qué es importante validar el tipo de dato al aplicar una condición con BETWEEN?

3

Uso de NOT IN

¿En qué casos usarías NOT IN en un entorno de negocio real?

4

Combinación de operadores

¿Cómo se pueden combinar AND y OR sin generar ambigüedades en la consulta?

5

Comparaciones de texto

¿Qué consideraciones debes tener al comparar campos de texto en SQL?

Referencias bibliográficas

- Coronel, C., & Morris, S. (2020). Database systems: design, implementation, & management (13.^a ed.). Cengage Learning.
- Hernández, M. J., & Getz, D. (2013). Designing relational databases. Addison-Wesley.
- Melton, J., & Simon, A. R. (2002). SQL: 1999: Understanding Relational Language Components. Morgan Kaufmann.
- Oppel, A. J. (2020). SQL: A Beginner's Guide (5.^a ed.). McGraw-Hill Education.
- PostgreSQL Global Development Group. (2024). PostgreSQL 15 Documentation. <https://www.postgresql.org/docs/>